

Service Index — Every Service, Every Role

The course teaches by use case, so the same service reappears in different jobs. This index inverts that: one entry per service, showing every role it plays and where.

How to use this: the exam rarely asks "what does X do". It asks "in *this* scenario, which service?" So read each row as **role → task**, and notice how one service's roles differ. Step Functions doing a circuit breaker (1.2) and Step Functions doing parallel processing (1.3) are the same service answering completely different questions.

The Multi-Role Services — learn these first

These six appear in the most places, in the most different roles. They cause the most confusion.

AWS Step Functions

Task	Role
1.2	Circuit breaker — detect model failure, route to fallback (closed / open / half-open)
1.3	Parallel multimodal processing — text, image, audio, tabular branches at once, then join
1.5	Parallel sub-query execution after query decomposition (–40% response time)
1.6	Conversation orchestration — state across turns, clarification loops; also edge case testing
2.1	ReAct pattern with your control; stopping conditions ; human approval via task token
2.3	Parallel and Map states ; error handling with SQS for reliable integration
3.1	Moderation workflow with risk-based branching; adversarial testing ; incident response
3.3	Model promotion approval workflow, gated on governance metrics
4.2	MaxConcurrency control to avoid throttling; parallel requests for latency
5.1	Continuous evaluation — daily tests against production
5.2	Sliding window processing; load shedding at peak

Pattern: Step Functions appears whenever you need *orchestration with state* — parallelism, waiting, branching, retries, or a pause for a human. Lambda alone cannot do any of those.

AWS Lambda

Task	Role
1.1	Query orchestration — retrieve, assemble prompt, invoke, emit event
1.2	Adapter — translate between your format and each model's format
1.3	Custom validation logic no managed service covers (VIN checks, procedure codes)
1.5	Query decomposition; batch embedding generation
2.1	Action group implementation behind an OpenAPI schema; stateless MCP server
2.2	Model <i>invocation</i> (not hosting); endpoint lifecycle orchestration
2.3	Protocol conversion (SOAP→REST); webhook handler with idempotency
3.1	Input sanitization; post-processing business-rule validation; Lambda authorizer
3.2	Data masking and anonymization
3.3	Automated compliance checks (rule engines); remediation actions
4.1 / 4.2	Pre-computation; recursive summarization; load testing frameworks
4.3	Scheduled index optimisation routines

Amazon CloudWatch

Task	Role
1.1	Bedrock metrics: InvocationLatency , token counts, throttles
1.2	Detecting degraded performance — custom metrics where error rate stays zero
1.6	KPIs including escalation rate — the subtlest signal
2.3	Composite alarms combining metrics; anomaly detection thresholds
3.1	Alarms on patterns of blocked content (abuse signal)
3.4	Per-demographic fairness metrics , including token counts per segment
4.1	Token consumption patterns; cache hit rates
4.3	Anomaly detection (learns baselines, understands seasonality); RUM ; Logs Insights
4.2 / 5.1	Evidently for A/B experiments with statistical analysis
5.2	Structured logging by error type/severity; Logs Insights for prompt confusion patterns

Amazon API Gateway

Task	Role
1.2	Standardised interface; custom authorizers ; mapping templates
2.3	Legacy format translation; usage plans per team ; response mappings
2.4	Request validators (JSON Schema); multi-level throttling (account/stage/route)
2.5	WebSocket APIs; chunked transfer encoding; content-type headers for streaming
3.1	Throttling as a safety control ; response filtering to prevent leakage
2.1	Feedback collection endpoints; exposing existing services as MCP endpoints

Amazon DynamoDB

Task	Role
1.4	Vector store option — sub-ms lookups, paired with Faiss/Pinecone (it does not do vector search alone)
1.6	Conversation history with TTL as a compliance control (90 days)
2.1	Tactical and strategic memory tiers; feedback storage with GSIs for pattern queries
2.3	Idempotency — storing processed event IDs
2.4	Current-state model performance metrics for routing
3.3	Approval tracking in governance workflows
4.1 / 4.2	Prompt component caching; pre-computed response storage

Amazon Comprehend

Task	Role
1.3	Extraction — entities, sentiment, language, key phrases from claim text
1.3	Normalisation — standardising dates and entities to one form
1.6	Custom classification — 200+ banking intents; cheap, fast, deterministic
3.1	Risk classification — pre-model filtering layer
3.2	PII detection (25+ types); Comprehend Medical for PHI
5.1	Sentiment analysis on unstructured user feedback

Bedrock — the feature map

Feature	Purpose	Main tasks
Knowledge Bases	Managed RAG — ingest, chunk, embed, store, retrieve. <code>Retrieve</code> vs <code>RetrieveAndGenerate</code> .	1.1, 1.4, 1.5
Prompt Management	Versioned, parameterised templates with approval workflows	1.1, 1.6, 3.4
Prompt Flows	Deterministic multi-step graph; conditional branching; no-code builder	1.1, 1.6, 3.4
Agents	Model chooses its own tool sequence (action groups = OpenAPI + Lambda)	2.1, 2.5
Guardrails	Content filters (4 categories × LOW/MED/HIGH), denied topics, PII, contextual grounding	1.6, 3.1, 3.2, 3.4
Model Evaluation	Automatic / LLM-as-judge / human; also toxicity and safety assessment	1.2, 3.1, 5.1
Provisioned Throughput	Dedicated capacity in Model Units; no throttling	2.2, 4.1, 4.2
Batch inference	Offline processing, roughly 50% cheaper	2.2, 4.1
Custom Model Import	SageMaker-trained model, Bedrock serverless pricing	2.2
Cross-region inference profiles	Distribute demand, reduce throttling	1.2, 2.2, 4.2
Model Invocation Logs	Full prompts and responses to S3/CloudWatch. Off by default.	1.1, 4.3
Agent tracing	<code>InvokeAgent</code> + tracing param — reasoning steps, retrieval scores, decisions	3.4, 4.3
Data Automation	Managed unstructured → structured extraction with validation checkpoints	2.5
Rerank models	Two-stage retrieval: top 100 → top 10, reading query and doc jointly	1.5
Streaming	<code>InvokeModelWithResponseStream</code> — perceived latency	1.1, 2.4, 4.2

SageMaker — the feature map

Feature	Purpose	Tasks
Pipelines	Standardised workflows; offline golden-dataset evaluation	1.1, 1.2
Model Registry	Versioning, lineage, <code>PendingManualApproval</code> gate	1.2, 3.3
Model Monitor	Drift: data quality, model quality, bias, feature attribution	1.2, 3.3, 5.2
Clarify	Bias metrics (DPPL, DI, DCR) and explainability; pre- vs post-training bias	3.3
ML Lineage Tracking	DAG of artifacts, actions, contexts; <code>AddAssociation</code> for cross-account	3.3
Model Cards	Purpose, limitations, biases, per-group performance; generated in CI/CD	1.2, 3.3, 3.4
Data Wrangler	Visual exploration — finds unknown unknowns in data	1.3, 4.3
Processing jobs	Custom transformation at scale; golden-dataset test runs	1.3, 4.3
Ground Truth	Expert annotation workflows	5.1
Inference Recommender	Automated load testing → optimal instance type	4.2
Feature Store	Time travel — point-in-time feature values for reproducibility	3.3
Endpoints	Real-time / serverless / async / batch transform; multi-model; inference components	2.2

Storage, Data & Retrieval

Service	Roles across the course
OpenSearch Serverless	1.4 vector store (hybrid search, fine-grained security) · 1.5 binary vectors, FP16, neural plugin · 4.2 HNSW/IVF tuning (<code>m</code> , <code>ef_construction</code> , <code>ef_search</code>), custom scoring · 4.1 semantic caching backend · 4.3 operational monitoring
Amazon S3	1.1 document source · 1.4 S3 Vectors option (cheap; <i>not</i> supported with SharePoint/Confluence/Salesforce sources) · 3.2 Lifecycle as a retention control · 3.3 versioning for reproducibility · 4.3 Macie scanning target
Aurora PostgreSQL + pgvector	1.4 / 1.5 — the only option that does SQL filtering and vector similarity in one query . Choose when structured metadata matters as much as similarity.
Neptune Analytics	1.5 — graph + vector together (<code>.vectors.topKByEmbedding</code>). For "similar to this <i>and</i> connected to that".
ElastiCache	1.4 pre-computed similarities · 2.1 operational memory tier (sub-ms) · 4.1 prompt component cache · 4.2 hot response cache
AWS Glue	1.3 Data Quality (DQDL rules — known unknowns) · 1.4 ETL for wikis and knowledge systems · 2.3 bulk sync with job bookmarks · 3.3 data lineage via <code>DynamicFrame</code> · 4.3 pre-index validation
Lake Formation	3.2 — column, row and cell-level security <i>inside</i> a table, where IAM only reaches the bucket
Amazon Macie	3.2 — discovers sensitive data <i>already sitting</i> in S3, with custom data identifiers
Amazon AppFlow	2.3 — SaaS connectors (Salesforce, ServiceNow), bidirectional, filtered
Timestream	2.4 — time-series model performance metrics driving routing decisions

Integration & Messaging

Service	Roles
EventBridge	1.1 fire-and-forget CRM notification (a synchronous call would make a CRM outage customer-facing) · 1.4 incremental vector updates · 2.3 pattern matching, input transformers, DLQ · 2.5 reactive half of hybrid orchestration · 3.3 triggering automated remediation · 1.5 EventBridge Pipes for indexing
Amazon SQS	2.2 absorb spikes, avoid throttling · 2.3 reliable ingestion, error handling · 2.3 FIFO with message group IDs (order within group, parallel across groups) · 4.1 average rather than peak capacity · 2.4 async FM requests with visibility timeout > processing duration
Amazon MQ	2.3 — only when the legacy system already speaks JMS/AMQP/MQTT. Active/standby for HA.
AppSync	1.1 GraphQL entry point; subscriptions for streaming; field-level auth · 2.3 VTL resolvers for real-time inference
Kinesis Data Streams	5.1 — high-volume real-time capture of user interaction patterns
MCP	1.5 the protocol itself · 2.1 agent-to-agent communication <i>and</i> building servers (Lambda for light, ECS for heavy) · 2.1 tool discovery

Security, Identity & Governance

Service	Roles
IAM	1.1 basic access · 2.1 least privilege on agent tools — the real security boundary, not the action group · 2.3 condition keys (source IP, time of day), permission boundaries · 3.2 model/feature/operation-level restriction, ABAC
Cognito	1.1 JWT validated locally against cached JWKS (no call per request) · 2.3 pre-token generation Lambda injecting enterprise roles · 1.4 matter-level permissions from signed claims
Verified Permissions (Cedar)	2.3 — application-level authorisation: "can you use it <i>on this document</i> ", where IAM only answers "can you call it"
SCPs / Organizations	3.3 — organisation-wide ceiling; never grants, only limits; account admins cannot bypass
KMS	2.3 multi-Region keys, rotation, grants · 3.2 encryption at rest with strict key policies
PrivateLink / VPC endpoints	2.2 private low-latency access · 2.3 endpoint policies restricting which operations · 3.2 Interface (most services) vs Gateway (S3, DynamoDB only)
AWS WAF	2.3 / 3.1 — rate-based rules matter uniquely here because every request costs money ; geo-match; injection protection
CloudTrail	1.6 which prompt, by whom, when (7-year retention) · 3.2 access auditing · 3.3 multi-region trails, data events off by default , Athena analysis
Security Hub / GuardDuty	2.3 / 3.1 / 3.2 — aggregated findings and threat detection across all layers
Audit Manager	3.4 — GenAI Best Practices Framework , automated evidence collection
Control Tower	2.3 — controls that block non-compliant deployments before they happen (data residency)
AWS RAM	2.3 — cross-account resource sharing with tag-based conditions

Networking & Edge

Service	Roles and the deciding factor
Global Accelerator	1.1 — anycast IPs, AWS backbone, failover in seconds. Enters only at the edge ; anycast routing happens before it. Two static IPs = redundancy, not one per region.
CloudFront	1.1 the alternative when content is cacheable · 4.1 / 4.2 edge caching for non-personalised responses
Outposts	2.3 — compliance : data legally cannot leave the premises
Local Zones	2.3 — distance : users concentrated far from the region
Wavelength	2.3 — mobile : inside the carrier 5G network, traffic never touches the internet
Direct Connect	2.3 — dedicated link; BGP policies prioritise inference over batch traffic
Transit Gateway	2.3 — domain isolation between dev/test/prod via route tables and black hole routes
Network Firewall	2.3 — outbound domain filtering so a compromised component cannot exfiltrate
EC2 Inferentia	4.2 — purpose-built inference silicon (Trainium is the training counterpart)

CI/CD, Config & Developer Tools

Service	Roles
AppConfig	1.2 externalised model selection — a settings file, not a place ; the Router reads from it · feature flags for gradual rollout and instant rollback · 2.4 static routing configuration
Parameter Store	2.4 — simpler alternative to AppConfig for plain key-value routing config
CodePipeline	1.2 model deployment with domain-specific gates · 2.3 model evaluation and security scanning stages · 5.1 automated quality gates that block releases
CodeBuild	1.2 accuracy / HIPAA / integration tests · 2.3 artifact caching (avoid re-downloading tens of GB)
CodeDeploy	2.3 — healthy host thresholds, long timeouts for LLM containers, CloudWatch-triggered rollback
CloudFormation	1.1 reusable IaC · 2.3 custom resources with DependsOn · 4.1 / 4.2 capacity planning templates
Service Catalog	2.3 — self-service deployment, but only from approved patterns (launch constraints)
Amazon Q Business	1.1 ready-made enterprise assistant · 2.5 data sources, refresh schedules (daily vs weekly), custom parsers
Amazon Q Developer	2.5 code generation with project context, refactoring prioritisation, test generation including prompt-injection cases · 2.5 / 4.3 error pattern recognition for GenAI failure modes
Amplify	2.3 DataStore with selective sync and conflict resolution · 2.5 UI with skeleton screens, progressive enhancement, ARIA
X-Ray	1.1 where the time went · 2.3 sampling rules (5% normal, 100% problematic) · 2.4 subsegments + business annotations · 5.2 found the bottleneck in auth middleware, not the model
QuickSight	3.3 automated compliance reports · 4.3 / 5.1 business-facing dashboards (CloudWatch is for engineers)
Cost Explorer / Budgets	4.1 attribute spend to teams and features · 4.3 Budgets for proactive threshold alerts

Reading the Index — Three Observations

- 1. Some services only ever do one thing.** Outposts is always compliance. Amazon MQ is always legacy protocols. Lake Formation is always in-table security. These are easy marks — recognise the signal, pick the service.
- 2. The multi-role services are where questions get hard.** Step Functions, Lambda, CloudWatch, Comprehend, DynamoDB and API Gateway each play five or more distinct roles. For these, the scenario's *problem* tells you which role applies — never the service name alone.
- 3. Several services are each other's alternatives.** Glue vs AppFlow, SQS vs MQ vs EventBridge, IAM vs SCP vs Verified Permissions, OpenSearch vs pgvector vs S3 Vectors. Those pairings are the discrimination sheet — study them there, not here.